

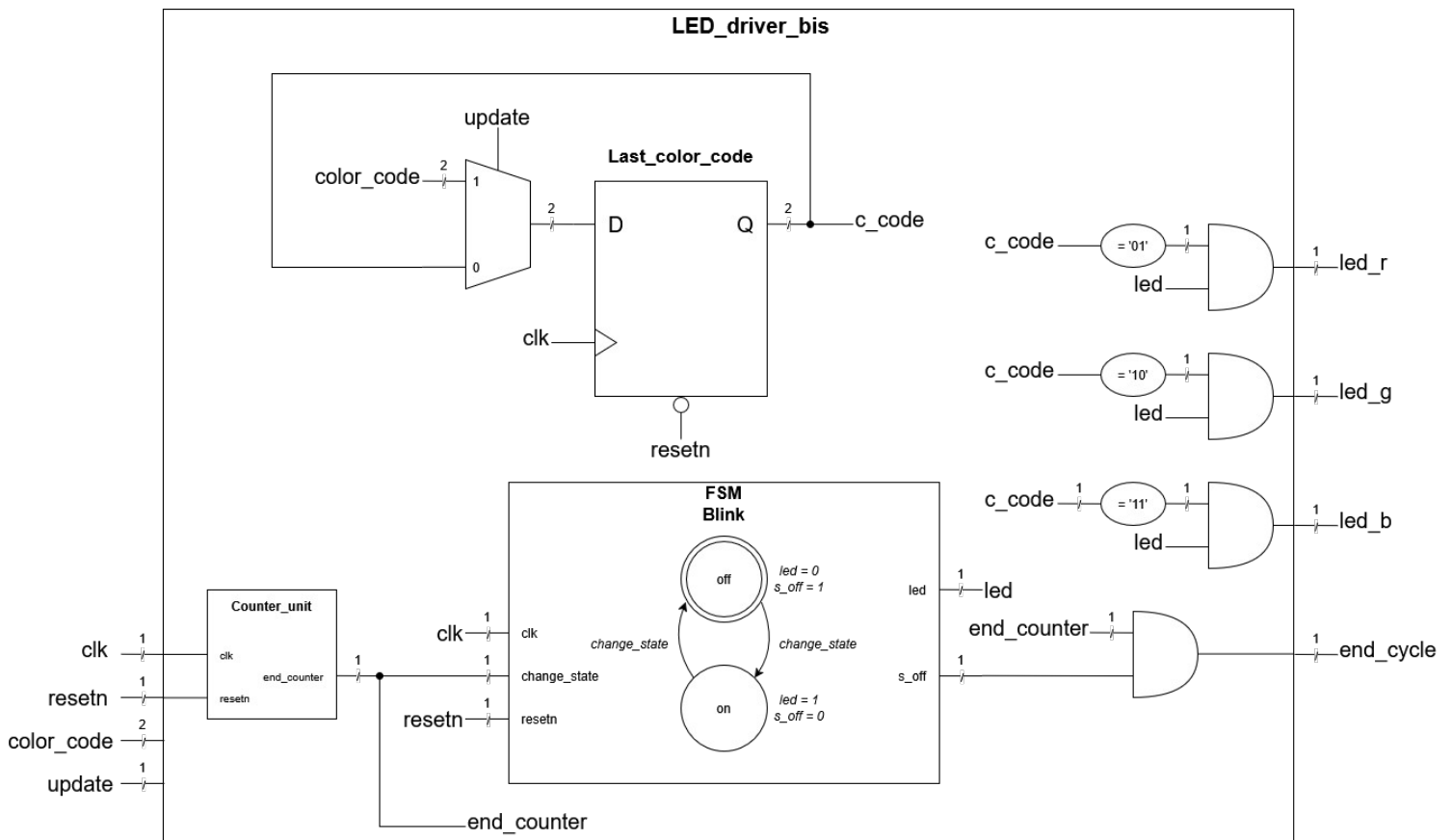
Formation FPGA

TP4 – Pilotage de LED et mémoire Partie 2

1) Nous modifions l'architecture RTL du module *LED_Driver* de la partie 1, pour ajouter une sortie *end_cycle*.

Pour cela, nous ajoutons une sortie *s_off* à notre FSM *Blink*, qui indique lorsqu'on est en phase éteinte. En combinant ce signal *s_off* avec le signal *end_counter* (qui était déjà un signal interne utilisé dans l'architecture, en sortie de *Counter_unit*) dans une porte AND, nous obtenons notre signal *end_cycle*, qui est activé pendant un coup d'horloge à la fin de la phase éteinte du clignotement.

Voici donc le nouveau schéma RTL correspondant à notre module modifié, renommé *LED_driver_bis* :



2) Nous modifions notre module *LED_Module* de la partie 1 pour y ajouter une FIFO.

Cette FIFO est pilotée par l'horloge globale *clk* et le signal de reset asynchrone *resetrn*.

Elle prend une entrée (*data_write*) à 2 bits, le signal en sortie du multiplexeur contrôlé par l'état du bouton 1, renommé *next_color_code* (anciennement simplement *color_code* dans la partie 1).

Cette entrée est contrôlée par une condition d'écriture dans la FIFO (*write_enable*) : le signal de front montant du bouton 0, renommé *update_fifo* (anciennement simplement *update* dans la partie 1).

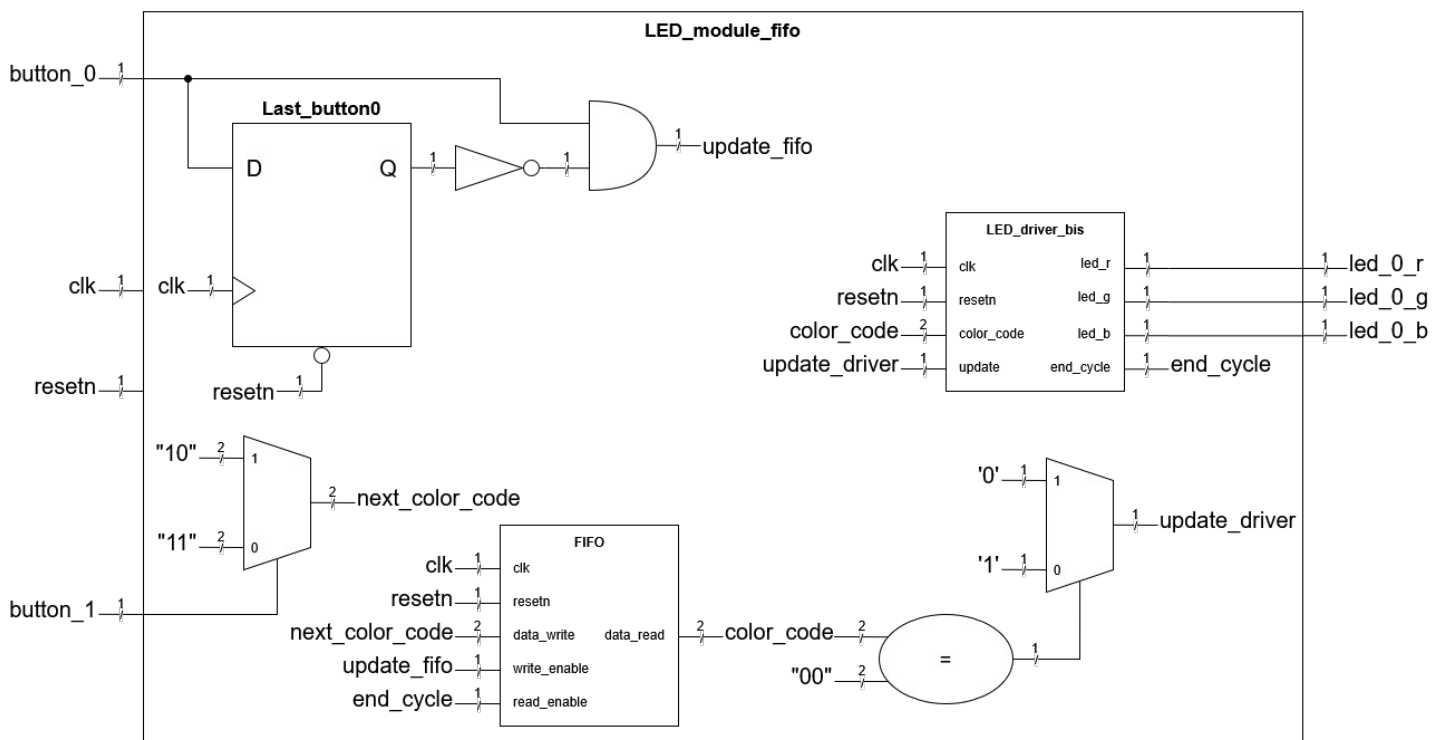
La sortie de la FIFO (*data_read*), toujours à 2 bits, est le nouveau signal *color_code* qui sert d'entrée au module *LED_driver_bis*.

Cette sortie est contrôlée par une condition de lecture de la FIFO (*read_enable*) : il s'agit du nouveau signal *end_cycle* en sortie de *LED_driver_bis*.

En plus de cette FIFO, une modification supplémentaire est nécessaire : le signal à l'entrée *update* de *LED_driver_bis*. Ce signal servait précédemment à sélectionner le moment où le code couleur est actualisé dans le module, mais cette sélection est maintenant accomplie dans la FIFO qui produit *color_code*. À la place, on va se servir de cette entrée pour garder le fonctionnement par défaut de *LED_driver_bis* avant la première *update_fifo*, qui clignote alors en rouge.

Puisque la FIFO renvoie un code '00' avant la première *update_fifo*, on crée un signal *update_driver* en sortie d'un multiplexeur : *update_driver* vaut '0' quand *color_code* vaut '00', et '1' sinon.

Voici donc le nouveau schéma RTL correspondant à notre module modifié, renommé *LED_module_fifo* :



En pratique, la FIFO utilisée pour notre module provient de l'IP Xilinx « FIFO Generator (13.2) », paramétrée de la manière suivante :

Estimated FIFO Usage : 1 of type: 1kx18

Number of FIFO chained in depth : 1

Clocking Scheme	Common Clock
Memory Type	Built-in FIFO
Model Generated	Behavioral Model
Write Width	2
Write Depth	1024
Read Width	2
Read Depth	1024
Almost Full/Empty Flags	Not Selected/Not Selected
Programmable Full/Empty Flags	Not Selected/Not Selected
Data Count Outputs	Not Selected
Handshaking	Not Selected
Read Mode / Reset	Standard FIFO / Asynchronous
Read Latency (From Rising Edge of Read Clock)	1

3) Nous implémentons les modifications faites à nos modules dans 2 nouveaux fichiers VHDL : *LED_driver_bis.vhd* et *LED_module_fifo.vhd*.

Le premier est modifié pour rajouter la logique combinatoire permettant de rajouter la sortie *end_cycle*.

Le second implémente l'instance de FIFO et la logique combinatoire créant le signal interne *update_driver*.

4) Nos modules modifiés sont testés dans un nouveau testbench, *tb_LED_module_fifo.vhd*.

On repart des paramètres de simulation utilisés pour le dernier testbench de la partie 1 :

- Une horloge de fréquence 100 MHz, soit une période de 10 ns, avec un signal carré (demi-période 5 ns)
- Une *cible* du compteur de 100000, ce qui correspond à des cycles de 1 ms

Pendant notre simulation, on va observer principalement :

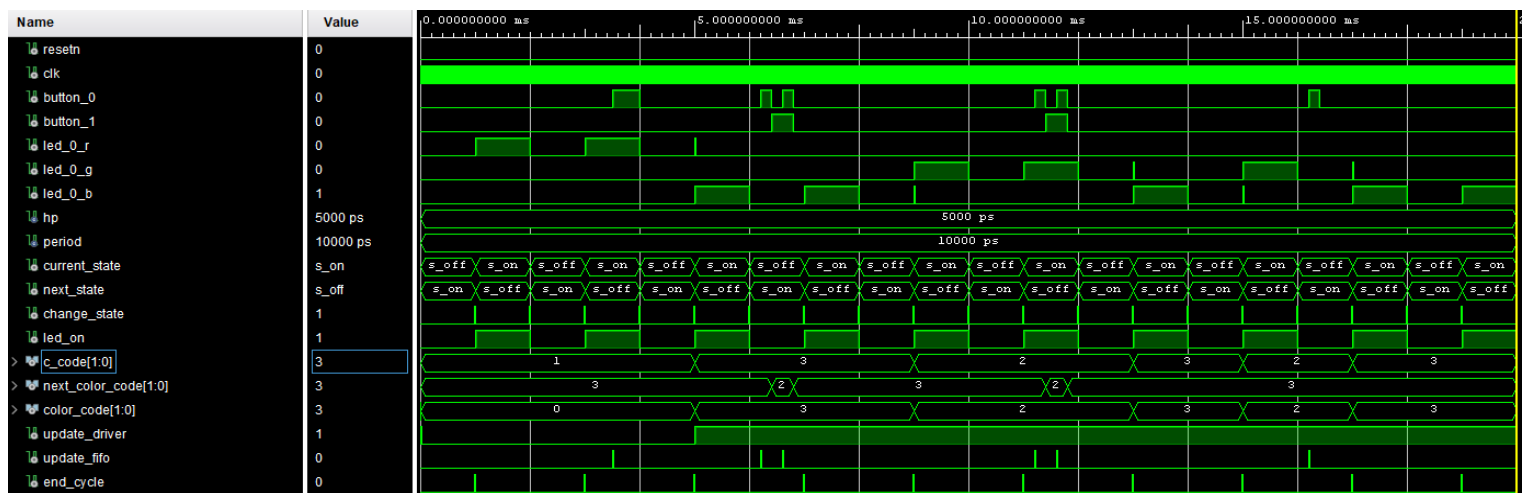
- Nos trois sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b*).
- L'entrée *button_0*, qui gère le signal *update_fifo*.
- L'entrée *button_1*, qui gère le signal *next_color_code*.

On va aussi observer quelques signaux internes pour vérifier le bon fonctionnement :

- Les signaux *current_state* et *next_state*, qui indiquent respectivement les états actuel et futur de la FSM *Blink*.
- Le signal *change_state* qui déclenche un changement d'état de la FSM *Blink* (lorsque le module *Counter_unit* atteint sa cible).
- Le signal *led_on* qui gère le clignotement global (pas la couleur).

- Le signal *c_code* qui indique la couleur actuelle dans laquelle la LED clignote.
- Les signaux *next_color_code* et *color_code*, respectivement en entrée et en sortie de la FIFO.
- Les signaux *update_fifo* et *end_cycle*, qui gèrent respectivement la mise-à-jour en écriture et lecture de la FIFO.
- Le signal *update_driver* qui active le changement de couleur du module *LED_driver_bis* après que la sortie de la FIFO *color_code* soit mise à jour pour la première fois.

Voici le chronogramme observé pour 20 ms de simulation, avec une activation du signal *button_0* à $t=3,5$ ms (sans bouton 1 à ce moment), puis à $t=6,2$ ms (sans bouton 1) et $t=6,6$ ms (avec bouton 1), ensuite à $t=11,2$ ms (sans bouton 1) et $t=11,6$ ms (avec bouton 1), et finalement $t=16,2$ ms (sans bouton 1) :



De manière générale, on observe que le mécanisme de clignotement introduit dans la partie 1 produit toujours un signal *led_on* qui alterne entre ses états bas et haut toutes les secondes, avec un signal *change_state* faisant un pic à chaque seconde, et l'état de la FSM (*current_state* et *next_state*) qui alterne après chacun de ces pics.

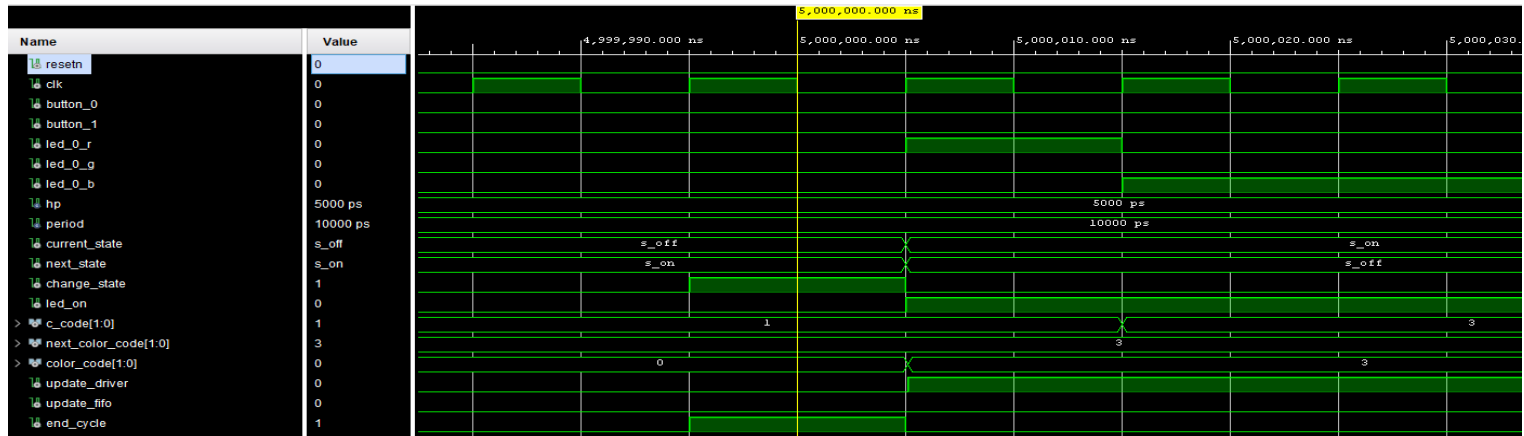
Jusqu'à $t=3,5$ ms, avant toute pression des boutons, on observe un clignotement de la LED rouge.

À $t=3,5$ ms, on appuie sur le bouton 0. Cela active le signal *update_fifo* : le code couleur à ce moment (bleu) est enregistré dans la FIFO. Rien d'autre ne se passe, le clignotement continue en rouge jusqu'à la fin du cycle normal, à $t=4$ ms.

À $t=5$ ms, on arrive à la fin de la période éteinte : le signal *end_cycle* s'active. À ce moment, deux processus vont se dérouler en parallèle (voir chronogramme détaillé en dessous) :

- Le signal *end_cycle* va lancer la lecture de la FIFO : le signal enregistré précédemment va être attribué à *color_code* (bleu). Au coup d'horloge suivant, c'est ce signal actualisé qui sera en entrée de *LED_driver_bis*, et qui va activer le signal *update_driver*.

- Dans *LED_driver_bis*, un nouveau cycle allumé commence, alors que *color_code* vaut à cet instant son ancienne valeur (rouge), et *c_code* reste inchangé. La LED s'allume donc en rouge. Au coup d'horloge suivant, *color_code* s'est actualisé en bleu, donc *c_code* va se mettre aussi à jour. La LED passe de rouge à bleu, et restera de cette couleur pour toute la période de clignotement.



Entre $t=6$ ms et $t=7$ ms, alors que la LED bleu s'est éteinte, on va appuyer deux fois sur le bouton 0, la première fois sans que le bouton 1 soit appuyé, et la deuxième alors qu'il est déjà appuyé. À chaque fois on voit le signal *update_fifo* s'activer : le code bleu puis le code vert sont ajoutés à la FIFO.

À $t=7$ ms, nouveau pic de *end_cycle*, la FIFO est lue et le code bleu est extrait. *color_code* était déjà bleu à ce moment, sa valeur ne change pas, un nouveau cycle allumé en bleu commence.

À $t=9$ ms, nouveau pic de *end_cycle*, la FIFO est lue à nouveau, cette fois le code vert est extrait. *color_code* change de valeur, et on observe à nouveau ce phénomène de l'allumage de la LED pendant un unique coup d'horloge de l'ancienne couleur (bleu) avant de prendre la nouvelle couleur (verte).

À nouveau, entre $t=11$ ms et $t=12$ ms, alors que la LED verte est allumée, on va appuyer deux fois sur le bouton 0, la première fois sans que le bouton 1 soit appuyé, et la deuxième alors qu'il est déjà appuyé. À chaque fois on voit le signal *update_fifo* s'activer : le code bleu puis le code vert sont ajoutés à la FIFO.

À $t=13$ ms, nouveau pic de *end_cycle*, la FIFO est lue à nouveau, le code bleu est extrait et *color_code* change de valeur. On observe la LED s'allumer pendant un unique coup d'horloge de l'ancienne couleur (verte) avant de prendre la nouvelle couleur (bleu).

À $t=15$ ms, nouveau pic de *end_cycle*, la FIFO est lue à nouveau, le code vert est extrait et *color_code* change de valeur. On observe la LED s'allumer pendant un unique coup d'horloge de l'ancienne couleur (bleu) avant de prendre la nouvelle couleur (vert).

Enfin, on appuie une dernière fois sur le bouton 0 (seul) à $t=16,2$ ms, alors que la LED verte est éteinte. Le signal *update_fifo* s'active : le code bleu est ajouté à la FIFO. Il est lu à $t=17$ ms, au pic

de *end_cycle* suivant, et on observe une nouvelle fois un unique coup d'horloge où la LED s'allume en vert avant de rester allumée en bleu.

Ce comportement d'allumage momentané de la LED de l'ancienne couleur lors de l'actualisation du code couleur interne n'est pas vraiment un problème car en pratique, 10 ns d'allumage sont indiscernable à l'œil humain.

Si on pilotait un signal plus sensible qu'une simple LED, se problème pourrait facilement être résolu en modifiant notre *Counter_unit* pour qu'il prévienne avec un autre signal un coup d'horloge avant la fin du compteur, et utiliser ce signal anticipé pour déterminer le signal *end_cycle* et l'actualisation du code couleur.

On a encore une fois implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement de notre module suit bien ce que nous attendons. Tous ces tests ont été vérifiés :

```
Note: test OK, only led_0_r = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, all led = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, only led_0_r = '1' at 3.5 ms
Time: 3500 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 pushed at 3.5 ms
Time: 3500 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: test OK, only led_0_r = '1' at 3.500010 ms
Time: 3500010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: update : button_0 released at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, all led = '0' at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, only led_0_r = '1' at 5.000010 ms
Time: 5000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, only led_0_b = '1' at 5.000020 ms
Time: 5000020 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, all led = '0' at 6.000010 ms
Time: 6000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: update : button_0 pushed at 6.2 ms
Time: 6200 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 released and button_1 pushed at 6.4 ms
Time: 6400 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 pushed at 6.6 ms
Time: 6600 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 and button_1 released at 6.8 ms
Time: 6800 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: test OK, only led_0_b = '1' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir

Note: test OK, only led_0_b = '1' at 9.000010 ms
Time: 9000010 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, only led_0_g = '1' at 9.000020 ms
Time: 9000020 ns Iteration: 0 Process: /tb_LED_module_fifo/lir
Note: test OK, only led_0_g = '1' at 11.000010 ms
Time: 11000010 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: update : button_0 pushed at 11.2 ms
Time: 11200 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 released and button_1 pushed at 11.4 ms
Time: 11400 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 pushed at 11.6 ms
Time: 11600 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 and button_1 released at 11.8 ms
Time: 11800 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: test OK, only led_0_g = '1' at 13.000010 ms
Time: 13000010 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: test OK, only led_0_b = '1' at 13.000020 ms
Time: 13000020 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: test OK, only led_0_b = '1' at 15.000010 ms
Time: 15000010 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: test OK, only led_0_g = '1' at 15.000020 ms
Time: 15000020 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: update : button_0 pushed at 16.2 ms
Time: 16200 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: update : button_0 released at 16.4 ms
Time: 16400 us Iteration: 0 Process: /tb_LED_module_fifo/line_
Note: test OK, only led_0_g = '1' at 17.000010 ms
Time: 17000010 ns Iteration: 0 Process: /tb_LED_module_fifo/li
Note: test OK, only led_0_b = '1' at 17.000020 ms
Time: 17000020 ns Iteration: 0 Process: /tb_LED_module_fifo/li
```

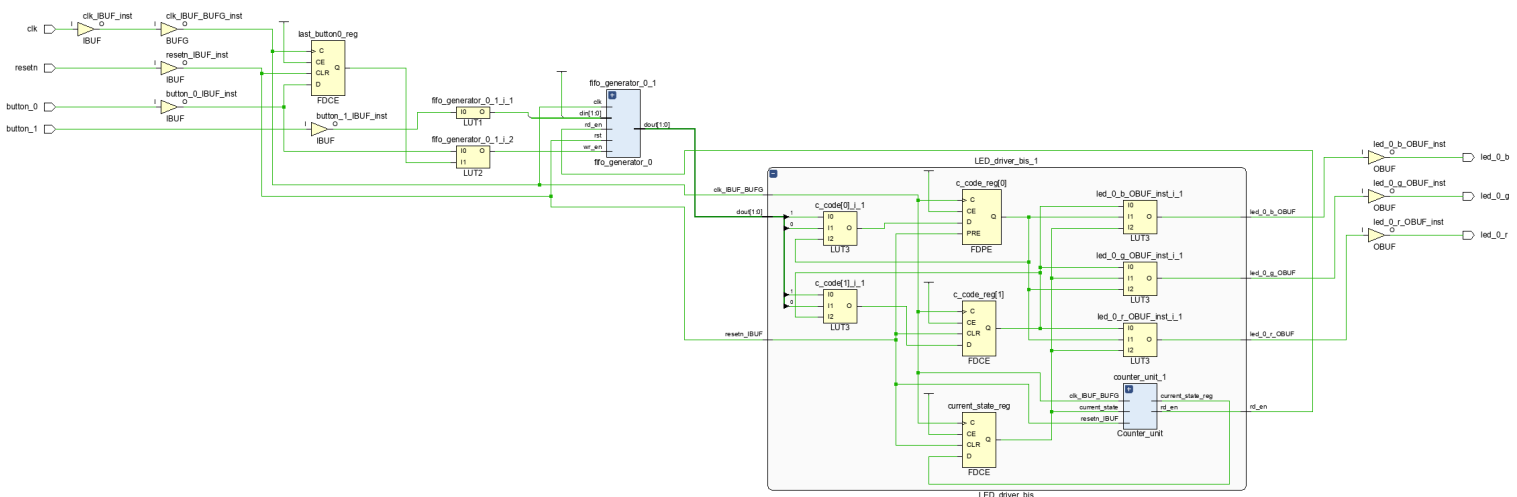
5) On réalise la synthèse de notre code sur Vivado. On notera que la synthèse de la FIFO s'est faite « out-of-context » lors de l'import de l'IP, seul le reste de notre module est donc synthétisé à cette étape.

Le rapport de synthèse (disponible dans son entièreté dans le fichier *LED_module_synth1.vds*) nous donne les ressources utilisées suivantes :

Report Cell Usage:

	Cell	Count
1	fifo_generator_0_bbox	1
2	BUFG	1
3	CARRY4	7
4	LUT1	1
5	LUT2	1
6	LUT3	19
7	LUT4	4
8	LUT5	2
9	LUT6	5
10	FDCE	30
11	FDPE	1
12	IBUF	4
13	OBUF	3

La schématique complète de notre synthèse est disponible dans le fichier *Schématique_TP4-2.pdf*.
Une version du schéma où le module *Counter_unit* n'est pas développé est proposée ci-dessous :



En plus de l'ajout de la FIFO, les modifications par rapport au schéma obtenu pendant la première synthèse de la partie 1 sont les suivantes :

- Deux tables de vérité LUT sont présentes en amont de la FIFO :
 - Une LUT1 permet d'obtenir le dernier bit de *next_color_code* en donnée d'entrée de la FIFO en fonction de l'état du bouton 1 (le premier bit vaut forcément '1').
 - Une LUT2 permet de détecter le front montant du bouton 0 pour créer le signal *update_fifo*.
- Deux tables de vérité LUT3 permettent maintenant l'actualisation des 2 bits de *c_code* à l'intérieur de *LED_driver_bis* (précédemment une LUT4 et une LUT3 qui prenaient en compte les entrées de boutons) : elles utilisent *color_code* (2bits) en sortie de FIFO et la précédente valeur de *c_code*, implémentant le test sur *color_code* correspondant au signal *update_driver*.

- Dans le module *Counter_unit* :
 - Un simili-signal *end_counter* est à nouveau créé, sur les sorties d'un couple de LUT6 (*fifo_generator_0_1_i_4* et *fifo_generator_0_1_i_5*). À noter que le bit 10 est toujours présent 2 fois dans la comparaison globale.
 - Ce couple de signaux *end_counter*, avec l'ancienne valeur de *current_state*, permettent encore de calculer le nouvel état de la FSM *current_state* en sortie d'une LUT3 (*current_state_i_1*).
 - Le couple de signaux *end_counter* est toujours en entrée de LUT3 pour déterminer la nécessité de réinitialisation de la plupart des bits du compteur valant '1' lorsqu'on atteint la *cible*.
 - Le dernier bit du compteur est encore utilisé, avec le couple de signaux correspondant à *end_counter*, dans 2 LUT3 pour déterminer la réinitialisation/incrémentation du bit et la propagation de l'incrémementation dans les bits supérieurs.
 - Les bits 12, 20 et 25 ont encore ce problème de redondance de comparaison, utilisant chacun une LUT6 et recalculant une partie du signal *end_counter* avec une LUT5 redondante (*Reg_ctr[12]_i_4*).
 - Le signal *end_cycle* est calculé par une LUT3, qui compare le signal *current_state* et le duo de signaux correspondant *end_counter*.

Le nouveau total de 32 LUT s'explique facilement en comparaison de la première synthèse de la partie 1 : 2 LUT supplémentaire en amont de la FIFO, et une LUT pour le calcul de *end_cycle*. Il n'y a aucune autre différence sur la quantité nécessaire pour les autres primitives.

À noter que cette synthèse projette l'utilisation de 27 « slice » de LUT, soit seulement 2 de plus que le module synthétisé dans la partie 1 (rapport d'utilisation *LED_module_fifo_utilization_synth.rpt*).

Avant de passer à l'implémentation dans la question suivante, on va également configurer les sondes ILA pour le debug de notre design.

On suivra les signaux suivants :

- Les 2 entrées des boutons 0 et 1 (*button_0_IBUF* et *button_1_IBUF*)
- Les 3 sorties LED (*led_0_r_OBUF*, *led_0_g_OBUF* et *led_0_b_OBUF*)
- Les 2 bits du signal *c_code* (*c_code[0]* et *c_code[1]*)
- Les 2 bits du signal *color_code* (*dout[0]* et *dout[1]*)
- Le dernier bit de *next_color_code* (*next_color_code[0]*)
- L'état de la FSM (*counter_unit_1_n_0*)
- Le signal de fin de cycle (*end_cycle*)
- Le signal de mise à jour en entrée de la FIFO (*update_fifo*)
- Les 2 signaux équivalents à *end_counter* (*fifo_generator_0_1_i_4_n_0* et *fifo_generator_0_1_i_4_n_0*)

Les contraintes liées à ce module de debug sont présentes dans le fichier *Cora-Z7-10-TP4-2-Debug.xdc*. Les contraintes liées au placement des autres entrées et sorties sur la carte sont dans le fichier de contraintes *Cora-Z7-10-Master-2.xdc*. À noter : en l'absence d'entrée dédiées pour le signal *resetn*, il a encore une fois été placé sur le premier pin du connecteur Pmod JA.

6) On effectue l'implémentation de notre résultat de synthèse.

Le processus d'implémentation ne soulève aucune erreur ou critical warning.

Analysons le rapport de timing (le rapport complet est disponible dans le fichier *LED_module_fifo_timing_summary_routed.rpt*).

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WFWS(ns)	TFWS(ns)	TFWS Failing Endpoints	TFWS Total Endpoints
1.747	0.000	0	4649	0.062	0.000	0	4633	2.750	0.000	0	2688

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 1,747 ns, ce qui est positif et indique qu'**il n'y a pas de violation de set up**.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,062 ns, ce qui est positif et indique qu'**il n'y a pas de violation de hold**.

Le récapitulatif d'horloge est le suivant :

Clock Summary			
Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
dbg_hub/inst/BSCANID.u_xsdbm_id/SWITCH_N_EXT_BSCAN.bscan_inst/SERIES7_BSCAN.bscan_inst/TCK	{0.000 16.500}	33.000	30.303
sys_clk_pin	{0.000 4.000}	8.000	125.000

L'horloge système *sys_clk_pin* est bien configurée selon nos spécifications initiales : période de 8 ns (soit une fréquence de 125 MHz), et signal carré (même demi-période de 4 ns pour les crêtes haute et basse du signal).

Le chemin critique présenté par le rapport est le suivant :

```

-----
From Clock: sys_clk_pin
To Clock: sys_clk_pin

Setup :          0 Failing Endpoints, Worst Slack      1.747ns, Total Violation      0.000ns
Hold  :          0 Failing Endpoints, Worst Slack      0.062ns, Total Violation      0.000ns
PW   :          0 Failing Endpoints, Worst Slack      2.750ns, Total Violation      0.000ns
-----

```

Max Delay Paths

```

-----
Slack (MET) :          1.747ns (required time - arrival time)
Source:          <hidden>
                  (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=8.000ns})
Destination:     <hidden>
                  (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=8.000ns})
Path Group:       sys_clk_pin
Path Type:        Setup (Max at Slow Process Corner)
Requirement:      8.000ns (sys_clk_pin rise@8.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay:  6.077ns (logic 0.642ns (10.564%) route 5.435ns (89.436%))
Logic Levels:     1 (LUT6=1)
Clock Path Skew:  -0.169ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD):    4.842ns = ( 12.842 - 8.000 )
  Source Clock Delay (SCD):         5.301ns
  Clock Pessimism Removal (CPR):    0.290ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ):        0.071ns
  Total Input Jitter (TIJ):          0.000ns
  Discrete Jitter (DJ):              0.000ns
  Phase Error (PE):                 0.000ns
-----

```

On remarque que le slack de ce chemin critique est à environ 22 % d'une période. C'est supérieur à la marge de 20 % qu'on veut avoir pour s'assurer d'un bon fonctionnement de notre système.

7) Sur Vivado, on génère le bitstream correspondant à notre implémentation, et on programme notre carte avec lui.

La vidéo *Démo_LED_module_fifo.mp4* présente 28 secondes de fonctionnement de la carte programmée.

La vidéo commence alors que la carte est dans son état initial (clignote en rouge).

Pendant les 2 premières secondes, on observe le clignotement en rouge de la LED.

À la 2^{ème} seconde, j'appuie rapidement quatre fois de suite sur le bouton 0 sans appuyer sur le bouton 1. 4 cycles de clignotement en bleu sont donc programmés.

À la 4^{ème} seconde, on voit la LED s'allumer en bleu (ce qui correspond à mon premier appui sur le bouton 0). En même temps, je réappuie 3 fois de suite sur le bouton 0, cette fois en gardant le bouton 1 enfoncé, ajoutant 3 cycles de clignotement en vert au programme.

À la 6^{ème} seconde, la LED s'allume une deuxième fois. Je relâche le bouton 1 pour appuyer 3 nouvelles fois sur le bouton 0, avant de réappuyer sur le bouton 1 et le bouton 0 une dernière fois, programmant 2 clignotements en bleu et un en vert.

Aux 8^{ème} et 10^{ème} secondes, la LED s'allume en bleu une troisième et une quatrième fois.

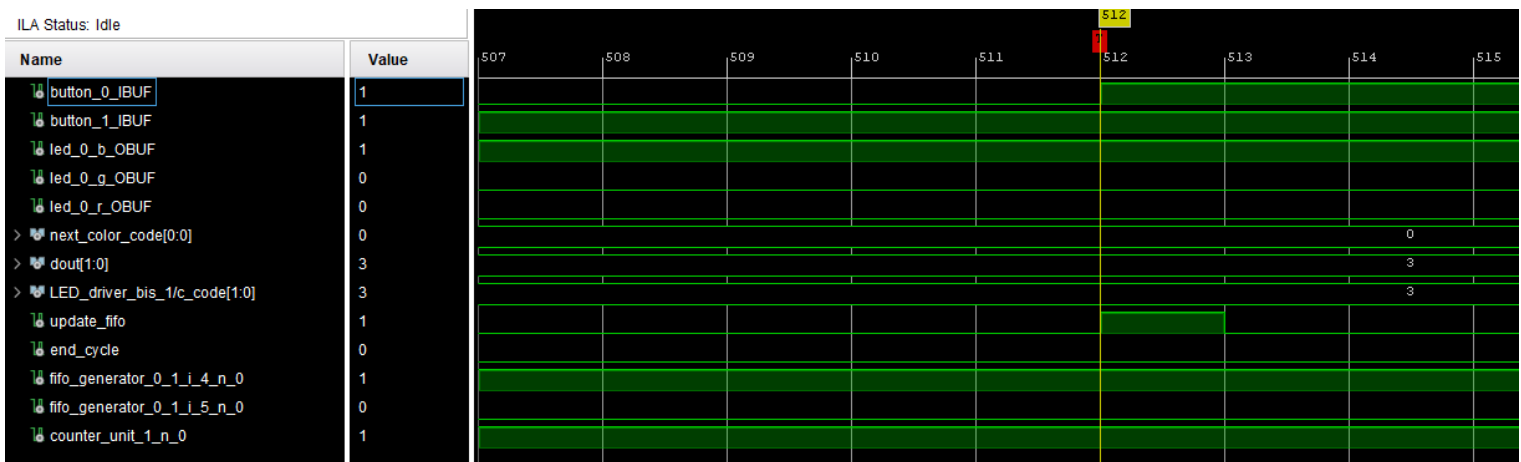
Aux 12^{ème}, 14^{ème} et 16^{ème} secondes la LED va clignoter en vert 3 fois.

Aux 18^{ème}, 20^{ème} et 22^{ème} secondes la LED va clignoter en bleu 3 fois.

À partir de la 24^{ème} seconde la LED clignote à nouveau en vert. En l'absence de nouvelle programmation de couleur enregistrée dans la FIFO, la LED continue de clignoter en vert jusqu'à la fin de la vidéo.

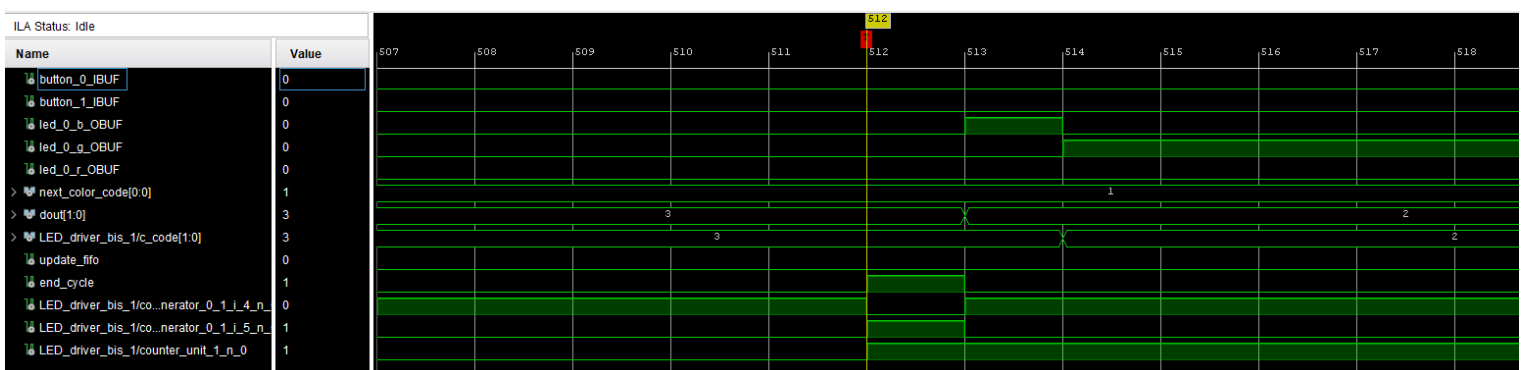
En plus de cette évaluation visuelle, on effectue plusieurs tests avec la ILA.

Le premier test vérifie le fonctionnement de l'enregistrement sur la FIFO, lorsque le bouton 0 est appuyé. Il est trigger lorsque *button_1* vaut '1' et qu'il y a un front montant sur *button_0*, pendant un clignotement bleu.



On observe bien que le front montant de *button_0* déclenche le signal *update_fifo*, qui doit lancer l'écriture de *next_color_code* dans la FIFO

Le second test vérifie le fonctionnement d'une fin de cycle, alors qu'un changement de code couleur en sortie de la FIFO a lieu. Il est trigger par *end_cycle* valant '1', le test étant lancé pendant clignotement bleu avec appui sur les 2 boutons.



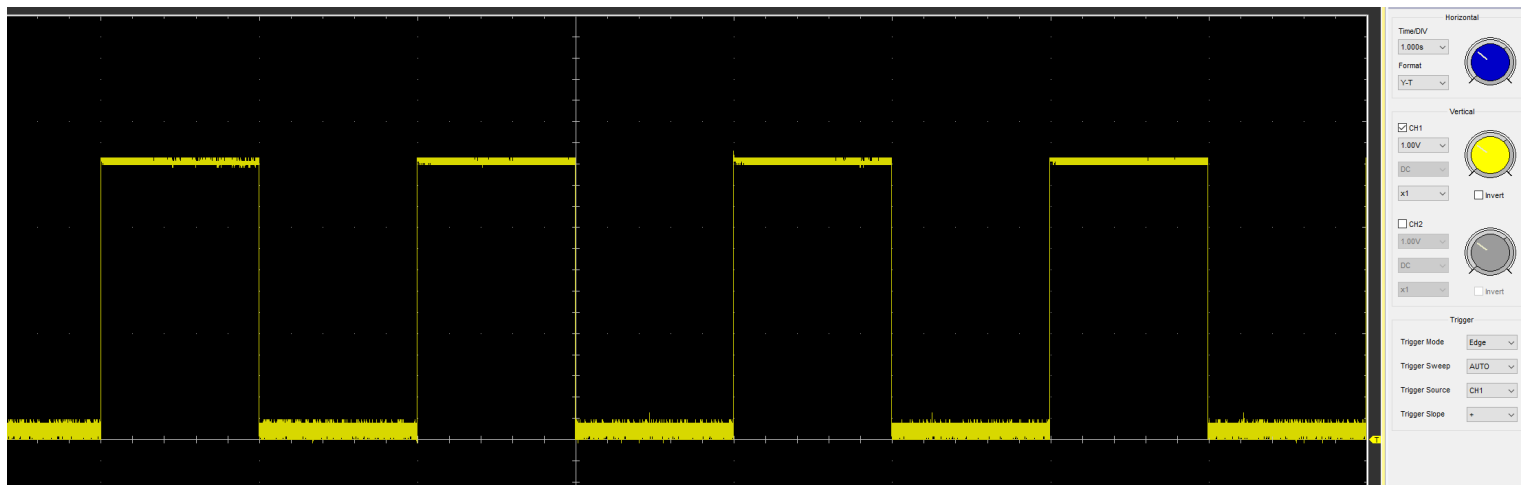
On vérifie bien que *end_cycle* est déclenché lorsque *end_counter* vaut '1' (*fifo_generator_0_1_i_4_n_0* vaut '0' et *fifo_generator_0_1_i_5_n_0* vaut '1') et que *current_state* vaut '0' (*counter_unit_1_n_0* vaut '0').

Comme lors de la simulation, on observe que l'allumage de la LED se fait pendant un coup d'horloge avec l'ancien code couleur, pendant lequel la sortie de la FIFO se met à jour (*dout[1:0]*), la LED reprend alors la couleur voulue.

Enfin, on vérifie également la durée de nos cycles avec une mesure à l'oscilloscope.

Notre mesure est effectuée au pin d'entrée de la LED bleue. Selon la documentation de notre carte Cora Z7, le signal de sortie de FPGA qui pilote les LED est inversé : lorsque le signal de sortie est haut, il est alors bas en sortie de la LED qui se retrouve sous tension, et donc allumée.

Nous mesurons le signal suivant (division verticale de 1 V, division horizontale de 1 s) :



On observe bien que lorsque la LED bleue est éteinte, le signal à l'entrée de la LED est haut, ce qui correspond à un signal de sortie bas, et inversement quand la LED est allumée. Notre carte semble avoir perdu un peu de puissance, puisque le signal haut monte vers 2,6 V, et non les 3,3 V prévu pour du LVCMOS33.

Sur cet enregistrement de notre signal, on observe que la période allumée de la LED, ainsi que la période éteinte entre deux clignotements valent bien 1 s à chaque fois. Cela confirme notre réglage du couple horloge – paramètre de *cible* de notre *Counter_unit* pour obtenir des cycles de 1 seconde.

Au final, que ce soit en visuel, via l'ILA ou à l'oscilloscope, la carte semble parfaitement fonctionner.